

Jamii Savings Integration Platform — Business Requirements Document

Author: Emmanuel · **Date:** 7 October 2026

1. Document control

Field	Value
Document	Business Requirements Document (BRD)
Project	Jamii Savings Integration Platform (WSO2 MI + APIM + Jenkins)
Version	1.4 (Final draft)
Author	Emmanuel
Target submission	12 Oct 2026 (hard deadline 14 Oct 2026)
Related documents	Solution Design Document, Jira project plan, presentation deck, demo recording, Git repository

Version	Date	Change
1.0	7 Oct 2026	Initial draft from assignment brief
1.1	7 Oct 2026	H2 database, all components as Docker images, ELK observability, self-managed WSO2 (Option C), nginx edge with jamii.daqiqtech.co.ke subdomains
1.2	7 Oct 2026	ELK and nginx moved to stretch goals to hold the 12 Oct submission; testing via MI unit tests and gateway-check.sh
1.3	7 Oct 2026	Backends replaced by a separately built and deployed Spring Boot mock service (customer REST and credit-check SOAP); Beeceptor, DNE Calculator and WireMock removed
1.4	8 Oct 2026	Aligned with the verified MI build: WSO2 MI and APIM 4.3.0 on JDK 17; two-layer log masking; MI smoke test 19/19

2. Executive summary

Jamii Savings needs a governed integration layer that exposes three core capabilities to digital channels and partners: account balance, customer profile, and loan eligibility. Each capability sits on a different backend: a relational database, a REST service, and a legacy SOAP service.

The solution uses WSO2 Micro Integrator (MI) to mediate and normalise these backends, WSO2 API Manager (APIM) to secure, throttle and publish them, and a single Jenkins pipeline to build, test and deploy everything consistently across environments.

The business outcome is one consistent, secure API contract for consumers, regardless of the backend technology behind it.

3. Business context and objectives

Jamii Savings is a fictional Kenyan savings bank serving retail customers in KES. Its channels (mobile app, internet banking, agents, partner fintechs) currently need point-to-point access to core systems, which duplicates logic, weakens security and slows delivery.

Business drivers

- **Channel enablement:** give mobile, web and partner apps a single, documented way to read balances, customer data and loan eligibility.
- **Legacy modernisation:** hide the SOAP credit system behind clean REST/JSON without rewriting it.

- **Governance and security:** every external call authenticated, throttled and auditable.
- **Data protection:** comply with the Kenya Data Protection Act 2019 by masking account numbers and PII in logs.
- **Speed and reliability of change:** automated, repeatable builds and deployments with no partial releases.

Measurable objectives

ID	Objective	Measure of success
BO-1	One consistent API contract	100% of error responses across the 3 APIs share one schema
BO-2	Secure exposure	0 APIs reachable on the gateway without a valid key or token
BO-3	Protect sensitive data	0 unmasked account numbers or PII in MI logs
BO-4	Controlled consumption	Throttling enforced per tier; excess calls return 429
BO-5	Repeatable delivery	One pipeline run builds, tests and deploys all 3 APIs, or fails cleanly with nothing half-deployed
BO-6	Self-service onboarding	A new consumer app can discover, subscribe and call an API from the Developer Portal without help
BO-7	Observable operations	Any request traceable end to end in Kibana by correlation ID; errors and latency per API visible on one dashboard

4. Scope

In scope (core)

1. Balance API: GET /accounts/{accountNumber}/balance, backed by a database via MI Data Services.
2. Customer API: GET /customers/{customerId}, a managed proxy in front of the Jamii mock customer service (Spring Boot).
3. Loan Eligibility API: POST /loans/eligibility, REST/JSON in front of the Jamii mock credit-check SOAP service (Spring Boot).
4. Cross-cutting: masked logging, input validation (400), and one standard error contract.
5. APIM: OpenAPI definitions, security, throttling tiers, one custom mediation policy, an API Product, Developer Portal onboarding guide.
6. CI/CD: one Jenkinsfile covering build, package, test, deploy to MI and import to APIM, with dev and prod stages.
7. Containerised runtime: every component (MI, APIM, H2) packaged as a Docker image and run with docker-compose.
8. Deliverables: Git repository, README, 5-minute demo recording.
9. Mock backends: a separate Spring Boot service (JDK 21, Maven, executable jar) that mocks the customer system and the credit-check engine, with its own Docker build and deployment.

Optional (bonus, only if core is complete)

- Bonus A: Customer dashboard aggregation API GET /customers/{customerId}/dashboard.
- Bonus B: Loan application event consumer with dead-letter queue.
- Stretch, started only after the core pipeline is green: ELK observability (6.7) and the nginx edge with jamii.daqiqtech.co.ke subdomains (6.8).

Out of scope

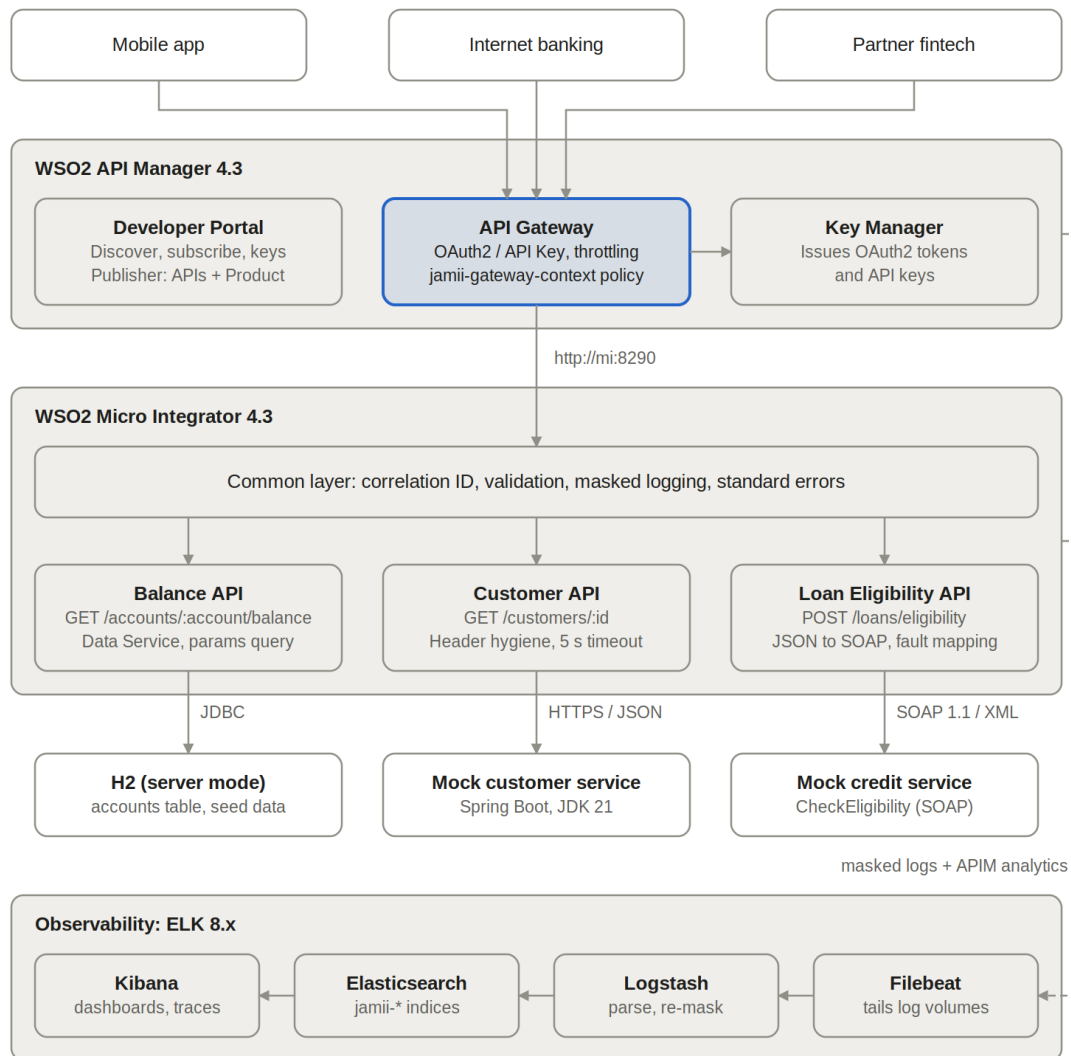
- Real core banking, credit bureau or production data.
- A live, permanently running hosted environment.
- Production identity provider integration (APIM's built-in Key Manager is used).

- High availability, clustering and performance testing at production volumes.

Solution context

Consumers reach the bank's capabilities only through the API gateway. Every component writes masked logs to a central ELK stack for operations and audit. ELK and nginx are stretch goals, delivered after the core pipeline is green.

Every consumer call passes nginx and the APIM gateway before it reaches MI



Logical architecture: consumers, APIM, MI, backends and ELK

The detailed component design is in the Solution Design Document.

5. Stakeholders and actors

Stakeholder / actor	Interest	Role in this project
Head of Digital Channels (business owner)	Faster channel features	Approves scope and acceptance
Retail customers	Accurate balances, fast loan answers	End beneficiaries via apps
Consumer applications (mobile app, internet banking, partner fintechs)	Stable, documented APIs	Discover, subscribe, call APIs
Integration team	Maintainable mediation	Build MI artifacts
API product owner	Governance, tiers, products	Configure and publish in APIM
Information Security / Data Protection Officer	Secure access, masked logs	Review security and logging

Stakeholder / actor	Interest	Role in this project
DevOps / Platform team	Reliable releases	Own pipeline and environments
Operations / Support	Fast troubleshooting	Use Kibana dashboards and correlation-ID traces
Interview panel (actual audience)	Evidence of competence	Evaluate repo, docs and demo

6. Functional requirements

Priority uses MoSCoW: M = Must, S = Should, C = Could. Every Must traces to a line in the assignment brief.

6.1 Balance API — GET /accounts/{accountNumber}/balance

ID	Requirement	Priority
FR-BAL-01	Retrieve the account record from the database using MI Data Services (datasource + query).	M
FR-BAL-02	Use parameterised queries only; no string-concatenated SQL.	M
FR-BAL-03	Return 200 with envelope: accountNumber, status (ACTIVE, DORMANT, CLOSED), balance {amount, currency: KES}, requestId (UUID), timestamp (ISO-8601).	M
FR-BAL-04	Return 404 with the standard error body when the account does not exist.	M
FR-BAL-05	Return a distinct error (503, code DB_UNAVAILABLE) when the database fails or is unreachable.	M
FR-BAL-06	Return 400 when the account number fails format validation.	M
FR-BAL-07	Ship a schema and seed script covering all three statuses, so the API is self-contained.	M

6.2 Customer API — GET /customers/{customerId}

ID	Requirement	Priority
FR-CUS-01	Proxy to the Jamii mock customer service (Spring Boot), with the choice justified in the README.	M
FR-CUS-02	Inject an X-Correlation-ID (UUID) when the caller does not send one; propagate it when they do.	M
FR-CUS-03	Strip internal headers (for example X-Internal-*, Server, X-Powered-By) from request and response.	M
FR-CUS-04	Apply a configured backend timeout; return 504 (BACKEND_TIMEOUT) when exceeded.	M
FR-CUS-05	Use a fault sequence to return 503 (BACKEND_UNAVAILABLE) when the backend is down.	M
FR-CUS-06	Map backend 404 to the standard 404 error; return 400 for an invalid customerId.	M

6.3 Loan Eligibility API — POST /loans/eligibility

ID	Requirement	Priority
FR-LOAN-01	Accept a JSON request and validate required fields and types; return 400 on failure.	M

ID	Requirement	Priority
FR-LOAN-02	Transform JSON to a SOAP envelope and call the Jamii mock credit-check SOAP service (CheckEligibility operation), with the choice and mapping justified in the README.	M
FR-LOAN-03	Transform the SOAP response into clean JSON with an eligibility decision.	M
FR-LOAN-04	Map SOAP Faults to the standard error contract (422 for business faults, 502 for server faults); never return SOAP XML to the client.	M
FR-LOAN-05	Handle SOAP backend timeout and unavailability with 504 / 503.	M

6.4 Cross-cutting (all three APIs)

ID	Requirement	Priority
FR-X-01	Log request in, backend call, and response out, each with the correlation/request ID.	M
FR-X-02	Mask account numbers and customer PII in all logs (for example *****7890).	M
FR-X-03	Return 400, not 500, for malformed input.	M
FR-X-04	Use one error response shape across all APIs regardless of backend (see section 8).	M

6.5 API Manager

ID	Requirement	Priority
FR-APIM-01	Publish all three APIs as managed APIs.	M
FR-APIM-02	Provide an accurate OpenAPI 3 definition per API, including error responses.	M
FR-APIM-03	Secure each API with OAuth2 or API Key.	M
FR-APIM-04	Assign throttling tiers; at least one API on a different tier, with a one-line reason.	M
FR-APIM-05	Apply at least one custom gateway mediation policy to one API.	M
FR-APIM-06	Group all three under one API Product, or justify not doing so.	M
FR-APIM-07	Document Developer Portal discovery and subscription for a consumer app.	M

6.6 CI/CD pipeline

ID	Requirement	Priority
FR-CI-01	One Jenkinsfile handles all three APIs (no copy-pasted pipelines).	M
FR-CI-02	Build MI artifacts with Maven and the WSO2 plugins into a composite application (CAR).	M
FR-CI-03	Package all three APIM APIs as an apictl project.	M
FR-CI-04	Run available tests (unit, contract, smoke).	M
FR-CI-05	Deploy CARs to MI and import/update APIs in APIM with apictl.	M

ID	Requirement	Priority
FR-CI-06	Separate dev and prod stages, with prod gated by approval.	M
FR-CI-07	Fail cleanly if any deployment fails; no silent partial deploy.	M
FR-CI-08	Package every component as a versioned Docker image (MI, APIM, H2, ELK); docker-compose runs the full stack, and the mock backends have their own image and compose file.	M

6.7 Observability (ELK), stretch

ID	Requirement	Priority
FR-OBS-01	Ship MI and APIM logs to Elasticsearch through Filebeat and Logstash.	S
FR-OBS-02	Parse each log line into fields: correlationId, api, stage, httpStatus, errorCode, latencyMs.	S
FR-OBS-03	Publish APIM analytics (ELK analytics mode) so gateway traffic, throttling and faults are indexed.	S
FR-OBS-04	Kibana dashboard: requests per API, error rate by code, p95 latency, 429 count.	S
FR-OBS-05	Trace one request end to end (APIM to MI to backend) in Kibana by correlation ID.	S
FR-OBS-06	Logstash re-masks any unmasked account-number pattern before indexing, as a safety net behind source masking.	S

6.8 Edge and access (nginx), stretch

ID	Requirement	Priority
FR-EDGE-01	nginx is the only public entry point and terminates TLS with a wildcard certificate (*.jamii.daqiqtech.co.ke; dev *.dev.jamii.daqiqtech.co.ke).	S
FR-EDGE-02	Route by hostname: api. gateway, portal. Developer Portal, apim. Publisher/Admin, mi. MI management API, kibana. Kibana, jenkins. Jenkins.	S
FR-EDGE-03	Restrict apim., mi., kibana. and jenkins. to an IP allowlist plus login.	S
FR-EDGE-04	Never publish MI runtime (8290), Elasticsearch, H2 or the image registry.	M

6.9 Bonus (optional)

ID	Requirement	Priority
FR-BON-A1	GET /customers/{customerId}/dashboard calls two or more Part 1 backends in parallel and merges results.	C
FR-BON-A2	Define and justify partial-response versus fail-whole behaviour.	C
FR-BON-B1	Consume a loan application event from a broker, transform it, and call a Part 1 API or publish onward.	C
FR-BON-B2	Route malformed messages to a dead-letter queue; explain async versus sync error handling.	C

7. Non-functional requirements

ID	Category	Requirement
NFR-01	Security	No API reachable on the APIM gateway without a valid OAuth2 token or API key; MI runtime port never exposed; all public traffic enters through nginx over TLS.
NFR-02	Security	No credentials committed to Git; secrets injected via Jenkins credentials and environment variables.
NFR-03	Data protection	Account numbers show last 4 digits only in logs; names, phone, email and ID numbers fully masked. A log-layout rule also masks any number of 10 or more digits in MI's own logs.
NFR-04	Performance	Backend timeouts: DB 5 s, Customer REST 5 s, SOAP 10 s. Gateway overhead target under 100 ms locally.
NFR-05	Resilience	Backend failure returns a controlled 5xx with the standard body within the timeout, never a hang or stack trace.
NFR-06	Observability	Every request carries a correlation ID end to end (client, APIM, MI, backend, response header). Logs are centralised in ELK and searchable by that ID.
NFR-07	Consistency	One error schema, one envelope style, ISO-8601 UTC timestamps, UUID v4 request IDs.
NFR-08	Maintainability	Shared logic (error builder, masking, correlation) implemented once as reusable sequences/templates.
NFR-09	Portability	Whole stack starts with one docker compose up; no host-specific paths; every service runs from a versioned Docker image.
NFR-10	Reproducibility	Pinned versions of MI, APIM, apictl, Maven plugins and DB images.
NFR-11	Documentation	README covers decisions, trade-offs, assumptions, run steps; OpenAPI specs match actual behaviour.

8. Data requirements

8.1 Account entity (Balance API source)

Field	Type	Rule
account_number	VARCHAR(20), primary key	Digits only, 10 to 16 characters
customer_id	VARCHAR(20)	Links account to customer
status	VARCHAR(10)	ACTIVE, DORMANT or CLOSED (CHECK constraint)
balance	DECIMAL(18,2)	Two decimal places
currency	CHAR(3)	Default KES
updated_at	TIMESTAMP	Last balance change

Stored in H2 running in server (TCP) mode in its own container. Seed data covers at least one account per status, plus a zero balance case.

8.2 Success envelope (Balance API)

```
{  
  "accountNumber": "0123456789",  
  "status": "ACTIVE",  
  "balance": { "amount": 15250.75, "currency": "KES" },  
}
```

```

"requestId": "5b1f8c2e-6a0d-4f7e-9b3a-2c1d0e9f8a77",
"timestamp": "2026-10-07T09:15:30Z"
}

```

8.3 Standard error contract (all APIs)

```

{
  "error": {
    "code": "ACCOUNT_NOT_FOUND",
    "message": "No account exists for the supplied account number.",
    "httpStatus": 404,
    "details": []
  },
  "requestId": "5b1f8c2e-6a0d-4f7e-9b3a-2c1d0e9f8a77",
  "timestamp": "2026-10-07T09:15:30Z"
}

```

details lists field-level validation problems for 400s; it is empty otherwise. No stack traces, SQL, hostnames or SOAP XML ever appear in an error body.

8.4 Data classification and masking

Data	Classification	Log treatment
Account number	Confidential	Last 4 digits only
Customer name, email, phone, address	PII	Fully masked
National ID / income in loan request	Sensitive PII	Fully masked
Balance amount	Confidential	Not logged
Correlation / request ID	Internal	Logged in full

9. Business rules and error handling matrix

The same failure produces the same HTTP status and error code on every API, whichever backend caused it.

Condition	HTTP	Error code	Balance (DB)	Customer (REST)	Loan (SOAP)
Malformed path, body or field	400	VALIDATION_ERROR	Bad account format	Bad customerId	Missing/invalid JSON fields
Missing or invalid credentials	401	UNAUTHORIZED	APIM	APIM	APIM
Resource not found	404	ACCOUNT_NOT_FOUND / CUSTOMER_NOT_FOUND	Query returns 0 rows	Backend 404	n/a
Business rule rejects request	422	ELIGIBILITY_REJECTED	n/a	n/a	SOAP client fault
Rate limit exceeded	429	THROTTLED	APIM	APIM	APIM
Unexpected internal error	500	INTERNAL_ERROR	Mediation error	Mediation error	Mediation error
Backend returned bad response	502	BAD_GATEWAY	n/a	Backend 5xx	SOAP server fault / unparseable
Backend unavailable	503	DB_UNAVAILABLE / BACKEND_UNAVAILABLE	DB down	Connection refused	Connection refused
Backend timeout	504	BACKEND_TIMEOUT	Query timeout	> 5 s	> 10 s

Business rules

- BR-01: A CLOSED account still returns its record and balance; status tells the consumer it cannot transact.
- BR-02: Currency is always KES for this release.
- BR-03: A caller-supplied X-Correlation-ID is kept; otherwise one is generated and returned in the response header.
- BR-04: Loan eligibility is advisory only; no loan is created.

10. Assumptions, constraints, dependencies and risks

Assumptions

- A-01: Jamii Savings is fictional; all data is synthetic.
- A-02: Account numbers are 10 to 16 digits; customer IDs are numeric.
- A-03: The customer system and credit-scoring engine are mocked by a purpose-built Spring Boot service, deployed separately; its contracts (customer JSON, credit-check XSD and WSDL) stand in for the bank's real systems.
- A-04: APIM's built-in Key Manager is the identity provider.
- A-05: Only the dev environment is physically wired; prod is a gated, conceptual stage.
- A-06: The jamii.daqiqtech.co.ke subdomains are the proposed target naming; locally they resolve through /etc/hosts and a mkcert certificate.

Constraints

- C-01: WSO2 MI and APIM 4.3.0 on JDK 17 (their start scripts accept JDK 11 to 17), both self-managed (Option C), apictl, Maven, Jenkins, Docker Compose, H2, Elastic Stack 8.x, nginx.
- C-02: Backends are self-built mocks rather than the existing public services the brief describes; the choice is justified in the README.
- C-03: APIM needs about 2 GB RAM and Elasticsearch about 1.5 GB; the full stack with ELK and Jenkins needs about 8 to 10 GB, so a 16 GB machine is the minimum.
- C-04: Submission by 12 Oct 2026 (hard deadline 14 Oct).

Dependencies

- D-01: Jamii mock backends service built, deployed and reachable from MI (shared Docker network or deployed host).
- D-02: Mock service contracts (customer JSON, credit-check XSD and WSDL) fixed before MI mapping starts.
- D-03: WSO2 product distributions (downloaded by the team) and Maven repositories available. D-04: DNS records and a wildcard certificate for jamii.daqiqtech.co.ke.

Risks

ID	Risk	Likelihood	Impact	Mitigation
R-01	Mock backends not running during the demo	Medium	High	Container health check; start the mocks first; backend URLs via config, not code
R-02	APIM and MI version incompatibility	Low	High	Pin matching versions; validate on Day 1
R-03	Laptop resources too low for full stack	Medium	Medium	Compose profiles start ELK and Jenkins only when needed; Elasticsearch and Logstash heaps capped
R-04	Time overrun on bonus items	Medium	Medium	Bonus only after core acceptance criteria pass
R-05	Secrets leaked in repo	Low	High	.gitignore, env files, Jenkins credentials store
R-06	Panel questions self-built mocks against	Medium	High	README justifies the choice: the mocks

ID	Risk	Likelihood	Impact	Mitigation
	the brief's 'existing public' wording			reproduce real failure modes (SOAP faults, delays, leaked headers) on demand, which public services cannot
R-07	WSO2 runtime rejects the JDK or misbehaves on version drift	Medium	High	Occurred and resolved: WSO2 images pinned to JDK 17 and 4.3.0 separately from other images; versions in .env

11. Acceptance criteria and traceability

The release is accepted when every Must requirement passes its test below and the evidence is in the repo or the recording.

Brief requirement	FR / NFR	Acceptance test	Evidence
1a DB-backed balance	FR-BAL-01 to 07	Valid account → 200 envelope; unknown → 404; DB stopped → 503 DB_UNAVAILABLE; abc → 400	MI unit test report, video
1b Customer proxy	FR-CUS-01 to 06	No header → correlation ID added; internal headers absent; backend blocked → 503; slow backend → 504	curl -v output, logs
1c SOAP to REST	FR-LOAN-01 to 05	Valid → clean JSON; SOAP fault → 422/502 JSON, no XML; bad JSON → 400	MI unit test report, video
Masked logging	FR-X-01, 02, NFR-03	grep logs: no full account number or PII	Log excerpt in README, Kibana search
Consistent errors	FR-X-03, 04, NFR-07	All error bodies validate against one JSON schema	MI unit test JSON-path assertions in pipeline
APIM publication	FR-APIM-01 to 03	Call without token → 401; with token → 200	gateway-check.sh (401/200), Developer Portal, video
Throttling tiers	FR-APIM-04	Exceed lower tier → 429	gateway-check.sh (429)
Mediation policy	FR-APIM-05	Policy effect visible in response or backend headers	curl output
API Product	FR-APIM-06	One subscription grants access to all three APIs	Developer Portal
Portal onboarding	FR-APIM-07	README steps followed by a new app succeed	README section
Single pipeline	FR-CI-01 to 05	One Jenkins run builds, tests, deploys all three	Jenkins console log
Environments	FR-CI-06	dev deploys; prod stage waits for approval	Jenkins stage view
No partial deploy	FR-CI-07	Break one API's import → pipeline fails, rollback runs, others not left half-updated	Jenkins log
Local stack	FR-CI-08, NFR-09	docker compose up brings up MI, APIM and H2 healthy from built images, with the mock backends already running (plus ELK and nginx if the stretch is delivered)	README, video

Stretch: observability (FR-OBS) is accepted when one request is traced by correlation ID in Kibana across APIM and MI, and the dashboard shows traffic, errors and latency per API.

Stretch: edge (FR-EDGE) is accepted when all six hostnames answer over TLS, admin hostnames refuse non-allowlisted IPs, and MI port 8290 is unreachable from outside the Docker network.

12. Glossary

Term	Meaning
MI	WSO2 Micro Integrator, the integration runtime that mediates backends
APIM	WSO2 API Manager: Publisher, Developer Portal, Gateway, Key Manager
CAR	Composite Application aRchive, the deployable MI bundle
apictl	WSO2 CLI to export, import and manage APIs and products across environments
Data Service	MI feature exposing database queries as services
Fault sequence	MI mediation flow that runs when an error occurs
Mediation policy	Custom logic applied at the APIM gateway on request or response
API Product	Bundle of resources from several APIs subscribed to as one
Throttling tier	Request quota applied per subscription
Correlation ID	ID that follows one request across every component for tracing
DLQ	Dead-letter queue for messages that cannot be processed
ELK	Elasticsearch, Logstash and Kibana (with Filebeat): central log collection, search and dashboards
nginx	Reverse proxy at the edge: TLS termination, hostname routing, IP allowlists
Mock backends	Separately deployed Spring Boot service that stands in for the customer system (REST) and credit-check engine (SOAP)